What We Built

APEX Semester Planning Agent

Live demo: https://technion-semester-palnner-agent.vercel.app

---

Short Version

We built a working semester-planning agent for Technion students. It has a frontend, required API endpoints, an offline data pipeline, a ReAct agent loop, planning tools, verification tools, optional memory, and transcript upload.

Main Components

Component	What was built
Frontend	A single-page web app with track selection, chat intake, plan display, transcript upload, cost display, and trace display.
Backend	FastAPI server with chat endpoints, streaming support, transcript parsing, session restore, and the required assignment endpoints.
Agent loop	A bounded ReAct loop where the model chooses tools and ends with `deliver_plan`.
Offline data pipeline	Scripts that fetch and package Technion catalog data, requirements, schedules, exams, CheeseFork reviews, and grade histograms.
Verification	Python checks for prerequisites, unknown courses, already-passed courses, duplicate courses, credit range, mandatory progress, schedule clashes, workload, and exam spacing.
Persistence	Optional Supabase session save and restore using a hashed student email.
Deployment	Vercel deployment with the root GUI and the required API routes.

Feature List

- Free-text planning in Hebrew or English.
- Three supported Technion tracks.
- Passed-course and failed-course handling.
- Failed-course retake priority.
- Grade-improvement retake analysis using the student's grade, course average, and the student's own average.
- Explicit add, remove, and swap requests.
- Course-question mode grounded in CheeseFork reviews.
- Section-level weekly timetable analysis.

- Exam calendar and exam-spacing checks.
- Workload score and risk report.
- Roadmap to graduation.
- Live offering check before final delivery.
- Transcript PDF parsing.
- Optional Supabase memory.
- Model architecture diagram endpoint.
- Step trace for the agent's reasoning loop.

What APEX Can Do - Examples

Real prompts against the live agent and the real text it returned - not invented examples.

Building a plan, resolving a conflict, and proposing a retake in one turn

Prompt: student starting semester 4, already passed "Human Factors in Data" (גורמי אנוש בדאטא) with a 55, class average 89.4.

"This is the best plan I found (7 courses, 18.5 credits) - I couldn't fully satisfy every constraint within this round. Still open: class time overlaps with [a humanities elective] in every section combination. Note: [that elective] was removed from the plan because its class times collide with another course in every available section combination - a clashing week is never delivered. Note: added [a mandatory course] - the plan had fallen below the credit floor; these fill it back out from the available shortlist. One more thing: your grade in גורמי אנוש בדאטא was notably below the comparison averages - want me to add a retake of it to your plan next time?"

Next turn, student replies "yes, add the retake":

"...Note: added your confirmed retake of גורמי אנוש בדאטא." The course now appears in the delivered plan tagged as a retake - never added without that explicit yes.

Refusing a course that doesn't exist, instead of silently dropping the request

Prompt: "add course 00000000 please" (not a real course number in the catalog).

"The added 00000000 can't be included because it isn't a valid course number, so that request is the only thing I had to reject."

Recognizing when nothing needs to change

Prompt: "Looks perfect, thank you!" - a plain closing remark on an already-delivered plan.

"Glad it works for you! Come back anytime you want to change something." No replanning runs at all - the agent recognizes this as an acknowledgment, not a change request, so the existing plan is returned untouched at a fraction of the usual cost.

Build Timeline

Stage	Result
Data collection	Built scripts for Technion data, CheeseFork reviews, and grade histograms.
Planner foundation	Built the first structured planning pipeline with draft, verify, repair, and explain stages.
Agent upgrade	Moved the main runtime to a ReAct loop with model-chosen tools.
Planning quality	Added reviews, exam spacing, workload, future impact, risk, and roadmap tools.
Reliability	Added deterministic guards for common failure cases and final delivery checks.
Product layer	Added frontend, transcript upload, Supabase memory, deployment, and assignment endpoints.

Testing

Test type	Purpose
Mocked tests	Check hard guarantees without spending model budget.
Architecture tests	Show the difference between the pipeline mode and the ReAct agent mode.
Revision tests	Check that follow-up requests preserve known state and do not restart unnecessarily.
Live tests	Exercise real model behavior on planning, retakes, constraints, and course questions.
Endpoint tests	Confirm the required Vercel routes respond with the expected shape.

Design Choice About RAG

We did not add a large RAG layer because the core task depends on exact structured facts. A vector search result should not decide whether a course is offered, whether prerequisites are met, or whether two classes overlap.

If the project grows, the best RAG addition would be a small policy search tool for academic regulations. That is separate from the planning core.

Why the Planning Prompt Is Long

The planner's system prompt embeds the full list of courses the student can still take, with difficulty, satisfaction, and exam date for each - because several planning rules (avoiding clustered exams, balancing workload, picking a well-reviewed elective) need to compare that data across candidates at once, not one course at a time. Fetching it per course instead would not shrink it: the same data would just move into tool-result history and add more model calls per plan. Input tokens are also priced well below output tokens, and measurement (not assumption) showed output length and

step count drive cost here, not prompt size - so prompt size was left as-is rather than trimmed for its own sake.

Final State

The project is an agent backed by a data pipeline. The pipeline prepares the knowledge. The runtime agent chooses tools, checks evidence, builds plans, repairs failures, and explains the result to the student.